Philippine Marching Percussion Fusion — AI and Development

AI and Development

Contents

Release: 2026.07

AI-assisted workflow and project development notes

0.1 Manual volumes in this release

PDF	Description
00-Trainers-Manual.pdf	Field teaching handbook for instructors and pilot collaborators
01-Specifications.pdf	Identity framework, blueprint, design principles, notation standards
02-Reference.pdf	Concept library, technique, grids, sound model
03-User-Guides.pdf	Instructor, pilot, arranging, and educational guidance
04-Research.pdf	Indigenous percussion, Kaamulan, marching heritage

PDF	Description
05-Architecture.pdf	Documentation architecture and ADRs
06-Governance.pdf	EDF governance, metadata, change management
07-AI-Development.pdf	AI-assisted workflow and project development notes
08-Templates-and-Placeholders.pdf	Templates and inactive EDF domain placeholders
09-Student-Practice-Guide.pdf	Student orientation and embedded technique reference

1 AI Engineering Handbook

Status: Maintained **Owner:** Philippine Marching Percussion Fusion Project **Applies To:** AI-assisted documentation, arranging, and project development **Last Reviewed:** 2026-07-11 **Review Frequency:** On Change **Authoritative:** Yes

This directory contains project guidance for using AI throughout the documentation, arranging, educational, and score-development workflow. AI tools can accelerate research, planning, documentation drafting, MuseScore preparation, and review when their roles are clear and project context is well organized.

1.1 Purpose

The AI Engineering Handbook defines:

- how AI tools support project work
- which tools are best suited for different tasks
- how to choose between local and cloud models
- how to control AI-related cost
- how to provide useful context to AI assistants
- how to verify AI-generated work
- how to preserve human accountability
- how AI assistants work directly on the local repository through Cursor (Composer 2.5 standard) or VS Code

1.2 Core Principle

Use the least expensive AI that can comfortably solve the task.

Project-owner productivity is part of the cost equation.

Saving a small amount of model usage cost is not worthwhile if it creates hours of additional effort, confusion, or rework — especially when working with musical notation, cultural context, or educational sequencing.

1.3 Handbook Documents

Document	Purpose
AI_Philosophy.md	Core principles for AI-assisted project work

Document	Purpose
AI_Roles.md	Responsibilities of Cursor + Composer 2.5, VS Code + Copilot, Continue, Ollama, cloud models, and the human project owner
AI_Decision_Matrix.md	Guidance for selecting the right AI tool for a task
Cost_Optimization.md	Strategy for balancing model cost, capability, and project-owner time
Prompting_Guide.md	Prompting practices for documentation and musical project work
Context_Checklist.md	Checklist for preparing useful AI context
Repository_Workflow.md	Standard workflow for AI-assisted work in the local repository
Verification.md	How to review and validate AI-generated work
Security.md	Security and privacy rules for AI usage
Governance.md	Human accountability and documentation governance

1.4 Parent

- Project Index *(Project Index (GitHub repository root))*

1.5 Related Documents

- PROJECT_CHARTER.md *(Project Charter (GitHub repository root; not in PDF manual))*
- ARCHITECTURE_DECISIONS.md *(not included in PDF manual; see project repository)*
- Documentation Architecture (Architecture)
- Project Governance (Governance)
- Core Blueprint (Specifications)

2 AI Philosophy

2.1 Purpose

This document defines the core philosophy for using AI tools within the Philippine Marching Percussion Fusion Project and other projects that adopt the Engineering Documentation Framework.

AI is treated as a project assistant, not an autonomous owner of musical quality, cultural authenticity, educational sequencing, or final creative decisions.

2.2 Primary Principle

Use the least expensive AI that can comfortably solve the task.

This does not mean always using the cheapest model. It means choosing the tool that provides the best overall engineering value after considering:

- task complexity
- required reasoning depth

- context size
- privacy requirements
- model capability
- developer time
- risk of rework
- cost of mistakes

Saving a small amount of API cost is not useful if it causes hours of additional project time or leads to poor creative or educational decisions.

2.3 Human Ownership

Human developers and project owners remain responsible for all final outcomes.

AI may assist with:

- brainstorming
- architecture analysis
- requirements refinement
- code generation
- documentation drafting
- testing suggestions
- debugging support
- review assistance

Humans remain responsible for:

- final engineering judgment
- security decisions
- production approval
- correctness verification
- maintainability
- testing
- release decisions

2.4 Documentation as AI Context

AI performance improves when the project has well-structured documentation.

The repository should be treated as the source of truth. AI assistants should be pointed to authoritative documents such as:

- `PROJECT_INDEX.md`
- `PROJECT_CHARTER.md`
- `ARCHITECTURE_DECISIONS.md`
- relevant specifications
- relevant architecture documents
- relevant user guides and educational documents
- `docs/AI/Repository_Workflow.md`

AI should not be given the entire repository blindly when a smaller, better-curated context is sufficient.

2.5 Same Standards as Human Work

AI-generated work must meet the same standards as human-written work.

AI-generated code and documentation should be:

- reviewed
- tested
- verified
- kept consistent with project architecture
- committed through the normal Git workflow
- documented when it affects architecture, behavior, or operations

2.6 Decisions Still Require Documentation

Significant architecture decisions influenced by AI still require formal documentation.

Use ADRs for decisions that affect:

- project structure and documentation architecture
- notation or production tooling choices
- educational methodology direction
- long-term maintainability

2.7 Related Documents

- AI_Roles.md
- AI_Decision_Matrix.md
- Governance.md

2.8 Parent

- AI Engineering Handbook

3 AI Repository Workflow

> **Status:** Maintained **Owner:** Philippine Marching Percussion Fusion Project **Applies To:** AI-assisted documentation, arranging, score preparation, and project development **Last Reviewed:** 2026-07-11 **Review Frequency:** On Change **Authoritative:** Yes

3.1 Purpose

This document defines the standard workflow for AI-assisted work on this Git-hosted music-education project.

AI assistants operate directly on the local repository working tree through an IDE such as **Cursor with Composer 2.5 (standard)** or **Visual Studio Code with GitHub Copilot**. The repository remains authoritative; changes are reviewed through Git and committed like any other project work.

3.2 Guiding Principle

The local Git repository is the single source of truth.

AI assistants must read and modify files in the current working tree. They must not rely on stale conversation copies, reconstructed content, or remote repository snapshots when the local repository is available.

3.3 Supported Environments

Environment	Role
Cursor + Composer 2.5 (standard)	Primary environment for documentation, arranging notes, and multi-file project work
Visual Studio Code + GitHub Copilot	Primary environment for developers using VS Code
VS Code + Continue + local models	Optional for privacy-sensitive or offline work

Browser-based AI tools that cannot access the local repository directly are not part of the standard workflow for this project.

3.4 Responsibility Model

3.4.1 AI Assistant Responsibilities

The AI assistant is responsible for:

- reading the current repository structure and relevant files from the working tree
- identifying the smallest focused set of required changes
- modifying complete files rather than supplying partial replacement fragments
- preserving repository paths and file names unless a change has been explicitly approved
- updating navigation and cross-links when documents change
- preserving Philippine musical identity in creative and educational content
- running available validation when practical
- reporting limitations, validation failures, and unresolved findings honestly

3.4.2 Project Owner Responsibilities

The project owner is responsible for:

- working in a current local clone of the repository
- reviewing the Git diff after AI-assisted changes
- running project-specific validation when needed
- accepting, revising, or rejecting the proposed changes
- committing approved changes in small, logical commits
- pushing completed work to the remote repository

The human project owner remains the final authority.

3.5 Standard Workflow

1. Open the project in Cursor (Composer 2.5 standard) or VS Code with GitHub Copilot.
2. Ensure the local repository is current (`git pull` when appropriate).
3. Identify the scope of the requested change.
4. Direct the AI assistant to read relevant files from the working tree.
5. The AI assistant modifies complete files in place.
6. Run the EDF Framework Advisor or other applicable validation when practical.
7. The project owner reviews `git status` and `git diff`.
8. The project owner commits approved changes.
9. The project owner pushes to the remote repository.

3.6 Complete File Delivery

When a file changes, the AI assistant must produce the complete replacement file at its normal repository path.

The AI assistant should not require the project owner to:

- copy and paste Markdown sections manually
- splice partial fragments into existing files
- reconstruct a document from multiple response blocks
- guess where a proposed change belongs
- reformat generated content before it can be reviewed

Complete-file delivery makes the proposed repository state explicit and allows Git to show the exact diff.

3.7 Small, Focused Changes

Each change set should address one focused implementation stage.

Do not combine unrelated work merely because several changes are available. Smaller changes provide:

- clearer Git diffs
- easier validation
- simpler rollback
- more meaningful commit history
- lower risk of accidental scope expansion
- better traceability between findings and corrections

3.8 Review Procedure

After AI-assisted edits, the project owner reviews the proposed changes:

```
git status
git diff
```

Run project validation before committing. From a local clone of the Engineering Documentation Framework repository:

```
/path/to/Engineering-Documentation-Framework/scripts/analyze_project_structure.sh \
  "/path/to/01_PH-Marching-Fusion-Project"
```

See Project Governance (Governance) for accepted analyzer exceptions affecting `scores/`, `audio/`, `references/`, and `reports/`.

After approval:

```
git add <changed-files>
git commit -m "Descriptive commit message"
git push
```

3.9 Repository Safety Rules

Unless explicitly approved for a particular change, the AI assistant must not automatically:

- overwrite unrelated user content
- rename user files
- delete user files
- move user files
- merge documents with uncertain ownership
- include unrelated cleanup
- change generated-file permissions without documenting the change
- treat an analyzer recommendation as authorization to modify the repository

Existing files become project-owned. EDF generators create only missing files. Analyzers remain read-only.

3.10 Verification

Whenever practical, the AI assistant should:

1. run the EDF Framework Advisor from a local EDF clone
2. capture the relevant pre-change findings
3. implement the focused corrections
4. rerun validation
5. report measurable improvements and remaining issues

A successful command does not replace human review. The Git diff remains the definitive view of the proposed change.

3.11 Commit Quality

Each commit should describe the implementation outcome, not the mechanics of AI assistance.

Preferred:

```
docs(ai): define repository-first IDE workflow
```

Avoid:

```
update files
```

3.12 Exceptions and Escalation

When the requested change cannot be completed safely from available repository information, the AI assistant should:

- complete all well-supported work that remains possible
- clearly identify what could not be completed
- avoid inventing repository contents
- avoid silently broadening the scope

3.13 Parent

- AI Engineering Handbook

3.14 Related Documents

- AI Philosophy
- AI Roles
- Context Checklist
- Verification
- AI Governance
- Project Governance (Governance)
- Documentation Change Management (Governance)
- Document Metadata Standard (Governance)

4 AI Roles

4.1 Purpose

This document defines the primary roles of AI tools within the Philippine Marching Percussion Fusion Project.

AI-assisted engineering happens directly in the local repository through IDE-integrated assistants. Different tools have different strengths. The goal is to select the tool that fits the task rather than using one model or product for everything.

4.2 Cursor + Composer 2.5 (standard) — Primary Repository Assistant

4.2.1 Primary Role

Cursor with **Composer 2.5 (standard)** is the primary AI environment for documentation and multi-file implementation work.

It operates directly on the local Git working tree and is best suited for coordinated changes across the repository.

4.2.2 Default Model

Use **Composer 2.5 (standard)** as the default Cursor model unless a task clearly requires escalation to a stronger reasoning model.

4.2.3 Typical Responsibilities

- reading and modifying documentation
- multi-file feature implementation
- large refactoring tasks
- cross-file updates
- project-wide code generation
- applying coordinated code changes
- test generation
- code modernization
- navigation and cross-link updates
- running validation and reviewing results

4.2.4 Best Used When

- implementing planned features
- modifying multiple related files
- updating documentation architecture
- performing project-wide changes
- working on EDF framework maintenance

4.3 Visual Studio Code + GitHub Copilot — Primary VS Code Assistant

4.3.1 Primary Role

GitHub Copilot provides AI assistance directly within Visual Studio Code on the local repository.

It is the primary AI tool for developers who work in VS Code rather than Cursor.

4.3.2 Typical Responsibilities

- inline code completion and suggestions
- explaining selected code
- answering programming questions within the open project
- refactoring individual functions or files
- writing unit tests
- debugging support
- interactive development assistance
- documentation drafting in context

4.3.3 Best Used When

- working within the currently open project in VS Code
- exploring unfamiliar code
- iterating on individual files or components
- asking implementation-specific questions with repository context

4.4 Visual Studio Code + Continue — Interactive Coding Assistant

4.4.1 Primary Role

Continue provides an interactive AI assistant directly within Visual Studio Code.

It is intended for focused development tasks, code understanding, and iterative problem solving while working inside a specific project.

Depending on task complexity, Continue may use either a local Ollama model or a cloud model such as Claude Sonnet.

4.4.2 Typical Responsibilities

- explaining code
- answering programming questions
- reviewing selected code
- refactoring individual functions
- writing unit tests
- debugging
- interactive development assistance

4.4.3 Best Used When

- working within the currently open project
- exploring unfamiliar code
- iterating on individual files or components
- asking implementation-specific questions
- privacy-sensitive work with local models

4.5 Local Ollama — Private, Low-Cost AI

4.5.1 Primary Role

Ollama provides local AI inference without requiring Internet access or API costs.

It is intended for routine development tasks where privacy, speed, or operating cost are more important than advanced reasoning.

4.5.2 Typical Responsibilities

- code explanation
- documentation
- JSON generation
- Markdown editing
- README updates
- small refactoring tasks
- learning unfamiliar code
- offline development

4.5.3 Advantages

- no API cost
- private execution
- works offline
- fast response time
- ideal for routine development support

4.5.4 Limitations

- smaller context window
- less capable reasoning than premium cloud models
- not recommended for large multi-file architecture or complex design problems

4.6 Claude Sonnet — Cloud Reasoning and Code Review

4.6.1 Primary Role

Claude Sonnet is a preferred cloud model for difficult coding, architecture, debugging, review, and cross-file reasoning tasks when used through an IDE-integrated assistant.

Use the latest generally available Claude Sonnet model unless a project-specific reason is documented for using a different version.

4.6.2 Best Used For

- architecture
- complex debugging
- cross-file reasoning
- code reviews
- large refactoring
- difficult implementation tasks

4.7 GPT-5 Reasoning — Exceptional Engineering Problems

4.7.1 Primary Role

GPT-5 reasoning should be reserved for exceptionally difficult engineering problems where deeper reasoning, careful trade-off analysis, or complex planning is justified.

4.7.2 Best Used For

- hard architecture problems
- complex debugging where cheaper tools failed
- high-impact design reviews
- deep trade-off analysis
- difficult project planning decisions

4.8 Codex — High-Value Agentic Implementation

4.8.1 Primary Role

Codex should be used only when its capabilities are justified by the value and complexity of the implementation task.

4.8.2 Best Used For

- high-value agentic implementation
- complex code generation
- coordinated changes with strong verification requirements

4.9 Human Developer — Final Engineering Judgment

4.9.1 Primary Role

AI assists the development process but does not replace engineering judgment.

The human developer remains responsible for all final technical decisions.

4.9.2 Responsibilities

- evaluate AI-generated recommendations
- verify correctness
- review security implications
- validate architecture
- approve implementation decisions
- ensure maintainability
- perform final testing
- accept responsibility for released software

AI should be viewed as an engineering assistant rather than an autonomous software engineer. Final responsibility for software quality, security, correctness, and long-term maintainability always remains with the developer.

4.10 Related Documents

- AI_Philosophy.md
- AI_Decision_Matrix.md
- Repository_Workflow.md
- Verification.md

4.11 Parent

- AI Engineering Handbook

5 AI Decision Matrix

5.1 Purpose

This document helps developers choose the appropriate AI tool for a software engineering task.

All work happens in the local repository through IDE-integrated assistants. The recommended approach is to start with the least expensive tool that can comfortably solve the problem, then escalate only when needed.

5.2 Roles Across the Software Development Lifecycle

SDLC stage	Primary AI	Secondary AI	Typical use cases
Vision and product ideas	Cursor + Composer 2.5	Claude Sonnet	Brainstorming, feature ideas, market analysis

SDLC stage	Primary AI	Secondary AI	Typical use cases
Requirements analysis	Cursor + Composer 2.5	Claude Sonnet	Functional requirements, user stories, acceptance criteria
Architecture and system design	Cursor + Composer 2.5	Claude Sonnet	System architecture, technology selection, trade-off analysis
Database design	Cursor + Composer 2.5	Claude Sonnet	Entity modeling, normalization, indexing strategies
API design	Cursor + Composer 2.5	Claude Sonnet	REST APIs, validation rules, security, versioning
Project planning	Cursor + Composer 2.5	Claude Sonnet	Roadmaps, milestones, task decomposition
Documentation	Cursor + Composer 2.5	VS Code + Copilot	Markdown, developer guides, user documentation
Single-file coding	VS Code + Copilot	VS Code + Qwen 14B	Explain code, small edits, documentation, local/private code
Multi-file coding	Cursor + Composer 2.5	VS Code + Copilot	Feature implementation, coordinated changes across multiple files
Complex implementation	Claude Sonnet	Cursor + Composer 2.5	Difficult algorithms, architectural refactoring, advanced debugging
Code review	Claude Sonnet	Cursor + Composer 2.5	Review design, identify bugs, improve maintainability
Testing	Cursor + Composer 2.5	Claude Sonnet	Unit tests, integration tests, test coverage
Deployment	Cursor + Composer 2.5	Claude Sonnet	CI/CD, deployment scripts, release documentation
Final engineering review	Human Developer	Cursor + Composer 2.5	Final design validation and engineering judgment

5.3 General Tool Selection

If you need to…	Recommended tool
Brainstorm products, requirements, architecture, APIs, or databases	Cursor + Composer 2.5
Explain or refactor a single file in VS Code	VS Code + GitHub Copilot
Work with sensitive or private code	VS Code + Continue + Qwen 14B
Understand multiple files or perform complex debugging	VS Code + Continue + Claude Sonnet
Implement features spanning multiple files	Cursor + Composer 2.5
Rapid multi-file implementation	Cursor + Composer 2.5
Perform code reviews	Claude Sonnet
Produce technical documentation	Cursor + Composer 2.5
Make the final engineering decision	Human Developer

5.4 Cursor + Composer 2.5 (standard)

Use Cursor with **Composer 2.5 (standard)** as the default model for repository work in Cursor.

5.4.1 Good Fit

- documentation updates
- multi-file implementation
- navigation and cross-link maintenance
- coordinated refactors
- framework maintenance
- validation-driven corrections

5.4.2 Advantages

- operates directly on the local working tree
- strong multi-file coordination
- good default balance of capability and cost for everyday engineering work

5.4.3 When to Escalate

Escalate to Claude Sonnet or another stronger model when Composer 2.5 has failed on a difficult problem or the task requires unusually deep reasoning.

5.5 VS Code + GitHub Copilot

Use GitHub Copilot when working in Visual Studio Code on routine development tasks with repository context.

5.5.1 Good Fit

- inline code completion
- explaining existing code
- learning an unfamiliar file
- small refactoring

- simple bug fixes
- unit test generation
- documentation within the open project

5.5.2 Advantages

- integrated with VS Code
- operates on the local working tree
- low friction for single-file work

5.5.3 Limitations

- weaker than agentic tools for large coordinated changes
- may struggle with complex multi-file architecture

5.6 VS Code + Qwen 14B

Use VS Code + Qwen 14B when working on routine, local, or privacy-sensitive tasks.

5.6.1 Good Fit

- explaining existing code
- learning an unfamiliar file
- writing Markdown
- creating JSON
- updating README files
- small refactoring
- simple bug fixes
- unit test generation
- working offline
- working with sensitive or proprietary code

5.6.2 Advantages

- free after installation
- runs locally
- fast response time
- no Internet required
- no API cost

5.6.3 Limitations

- smaller context window
- weaker reasoning
- not suitable for large multi-file projects
- may struggle with complex architecture

5.7 VS Code + Claude Sonnet

Use VS Code + Claude Sonnet when stronger reasoning or broader context is required.

5.7.1 Good Fit

- working across multiple files
- understanding a large codebase
- designing new features
- performing architectural refactoring
- debugging difficult problems
- reviewing pull requests
- evaluating design alternatives
- implementing complex business logic

5.7.2 Advantages

- excellent reasoning
- strong architectural understanding
- large effective context window
- high-quality code reviews

5.7.3 Trade-Offs

- requires Internet access
- API usage costs money
- should be reserved for problems where the additional capability provides clear value

5.8 Escalation Strategy

Start with the lowest-cost tool that is likely to solve the task successfully.

Recommended progression:

1. VS Code + GitHub Copilot or Qwen 14B for routine single-file work
2. Cursor + Composer 2.5 (standard) for multi-file documentation and implementation
3. Claude Sonnet for stronger reasoning, large context, and code review
4. GPT-5 reasoning for exceptionally difficult engineering problems
5. Codex only when high-value agentic implementation is justified

5.9 Related Documents

- AI_Roles.md
- Repository_Workflow.md
- Cost_Optimization.md
- Context_Checklist.md

5.10 Parent

- AI Engineering Handbook

6 Context Checklist

Status: Maintained **Owner:** Philippine Marching Percussion Fusion Project **Applies To:** AI-assisted project work **Last Reviewed:** 2026-07-11 **Review Frequency:** On

Change

6.1 Purpose

This document defines the information that should be provided to AI assistants when performing project work.

The goal is to provide enough context for useful output without overwhelming the assistant with unrelated files.

6.2 Core Checklist

Before asking AI to perform project work, identify:

- relevant requirements and specifications
- applicable ADRs
- relevant architecture documents
- musical, educational, or arranging constraints
- only the files required for the task
- clear success criteria
- known constraints
- expected output format
- whether the work is planning, editing, composition, documentation, review, or troubleshooting

6.3 Repository Entry Points

When starting a new AI session, useful entry points include:

- `PROJECT_INDEX.md`
- `PROJECT_CHARTER.md`
- `ARCHITECTURE_DECISIONS.md`
- `docs/Specifications/Core_Blueprint.md`
- `docs/Reference/Concept_Library.md`
- `docs/Architecture/`
- `docs/Specifications/`
- `docs/User_Guides/`
- `docs/AI/`
- `scores/` — when working on notation or warm-ups
- `references/` — when working on groove vocabulary or inspiration

6.4 Avoid Overloading Context

Do not provide the entire repository unless the task genuinely requires it.

Prefer targeted context:

- the affected file
- related specification or concept-library entry
- related architecture document
- relevant score or audio deliverable
- relevant ADRs

6.5 For Documentation Tasks

Provide:

- target document path
- purpose of the document
- audience (educator, arranger, performer, contributor)
- related documents
- preferred tone
- whether to create, update, split, merge, or review

6.6 For Musical and Arranging Tasks

Provide:

- target instrument section or score path
- groove vocabulary or cultural constraints
- notation standards in effect (or note if still planned)
- MuseScore or playback constraints
- layering intent (Philippine groove, DCI overlay, HBCU contrast)
- acceptance criteria for musical identity and teachability

6.7 For Architecture Tasks

Provide:

- project goals
- constraints
- current documentation architecture
- alternatives considered
- known risks
- affected domains or deliverable areas

6.8 Related Documents

- Prompting_Guide.md
- AI_Decision_Matrix.md
- Verification.md
- Core Blueprint (Specifications)

6.9 Parent

- AI Engineering Handbook

7 Prompting Guide

7.1 Purpose

This document provides practical prompting guidance for engineering work.

Good prompts reduce ambiguity, improve AI output quality, and help keep documentation and implementation aligned.

7.2 General Prompting Principles

- Start with the smallest amount of context necessary.
- Reference `PROJECT_INDEX.md` and relevant specifications.
- Identify the target files or folders when possible.
- State the desired output format.
- Ask for trade-off analysis before implementation.
- Keep documentation and code synchronized.
- Verify all AI-generated output before committing.

7.3 Provide Clear Context

When asking an AI assistant to work on a project, include:

- project goal
- relevant files
- current problem
- expected outcome
- constraints
- desired style or format
- whether the assistant should edit, review, summarize, or plan

7.4 Prefer Specific Instructions

Weak prompt:

```
Fix the docs.
```

Better prompt:

```
Review PROJECT_INDEX.md and README.md. Update them so they reference docs/AI/ as the authorita
```

7.5 Ask for Planning Before Large Changes

For complex work, ask the AI to summarize its plan first.

Useful pattern:

```
Before editing, inspect the current repository structure and summarize the files you intend to
```

7.6 Require Preservation of Existing Content

When refactoring documentation, explicitly require that existing content be preserved unless intentionally removed.

Useful pattern:

```
Refactor this document into smaller files. Do not lose information. If content is removed, expl
```

7.7 Use Documentation Domains

When asking AI to create or move documentation, reference the framework domains:

- Architecture

- AI
- Development
- Specifications
- API
- Database
- Deployment
- User Guides
- Reference
- Templates
- Archive

7.8 Recommended Workflow Prompts

7.8.1 New Feature

1. Define requirements in the local repository.
2. Review architecture with Cursor (Composer 2.5) when needed.
3. Record important decisions in `ARCHITECTURE_DECISIONS.md`.
4. Implement with Cursor (Composer 2.5) or VS Code + GitHub Copilot.
5. Escalate to Claude Sonnet only when stronger reasoning is required.
6. Review, test, and commit.

7.8.2 Existing Code Investigation

1. Begin with VS Code + Copilot or Continue + local AI.
2. Escalate to Claude Sonnet for multi-file reasoning.
3. Document important findings in the repository.

7.8.3 Documentation

1. Draft with Cursor (Composer 2.5) in the local working tree.
2. Verify against the implementation.
3. Commit documentation with related code whenever practical.

7.9 Related Documents

- Context_Checklist.md
- Verification.md
- Governance.md

7.10 Parent

- AI Engineering Handbook

8 Verification

8.1 Purpose

This document defines how AI-generated work should be reviewed and validated.

AI output should never be accepted blindly. It must be verified according to the same standards as human-generated work.

8.2 General Verification Rules

AI-generated work should be checked for:

- correctness
- completeness
- security implications
- consistency with architecture
- consistency with documentation
- maintainability
- test coverage
- formatting and style
- broken links
- unintended file changes

8.3 Code Verification

When reviewing AI-generated code:

- read the diff carefully
- run relevant tests
- inspect error handling
- verify data validation
- check security-sensitive logic
- confirm naming and style consistency
- ensure no secrets were introduced
- verify dependency changes
- confirm performance implications when relevant

8.4 Documentation Verification

When reviewing AI-generated documentation:

- confirm facts against the implementation
- verify links
- check that authoritative sources remain clear
- remove duplication
- ensure terminology is consistent
- update `PROJECT_INDEX.md` when major documents are added or moved
- ensure outdated content is archived or clearly replaced

8.5 Architecture Verification

When AI influences architecture:

- review trade-offs
- document significant decisions in ADRs
- confirm alternatives were considered

- validate assumptions
- check long-term maintainability
- confirm operational implications

8.6 Git Verification

Before committing AI-generated changes:

- inspect `git status`
- review the full diff
- confirm no unrelated files changed
- confirm no files were accidentally deleted
- confirm generated files are intentional
- run tests when applicable

8.7 Related Documents

- Governance.md
- Security.md
- AI_Philosophy.md

8.8 Parent

- AI Engineering Handbook

9 Security

9.1 Purpose

This document defines security and privacy rules for using AI tools in software engineering projects.

AI usage must not compromise credentials, regulated data, private information, intellectual property, or production systems.

9.2 No Secrets in Prompts

Never paste secrets into cloud AI tools.

This includes:

- API keys
- passwords
- private keys
- database credentials
- access tokens
- production configuration values
- customer data
- regulated data
- personally identifiable information unless explicitly approved by policy

9.3 Use Local AI for Sensitive Context

When working with sensitive or proprietary code, prefer local AI tools when practical.

Local AI is useful for:

- private code explanation
- internal documentation
- small refactoring
- offline work
- sensitive project context

9.4 Cloud AI Approval

Cloud AI may be appropriate for many engineering tasks, but teams should define what may and may not be shared.

Projects should document:

- approved AI tools
- approved data categories
- prohibited data categories
- model usage policies
- review requirements
- exception process

9.5 Security Review

AI-generated changes require extra care when they affect:

- authentication
- authorization
- encryption
- input validation
- database access
- file handling
- network communication
- deployment configuration
- secrets management
- audit logging

9.6 Documentation Security

Documentation should never expose secrets.

Deployment and environment documentation may list variable names, but not secret values.

Example acceptable documentation:

```
DATABASE_URL must be configured in the deployment environment.
```

Example unacceptable documentation:

```
DATABASE_URL=postgres://user:password@example.com/db
```

9.7 Related Documents

- Governance.md
- Verification.md
- AI_Decision_Matrix.md

9.8 Parent

- AI Engineering Handbook

10 AI Governance

10.1 Purpose

This document defines governance expectations for AI-assisted engineering work.

AI can accelerate documentation, arranging, and educational development, but human project owners remain accountable for all outcomes.

10.2 Human Accountability

Human developers are responsible for:

- final engineering judgment
- security approval
- production approval
- architectural ownership
- accepting or rejecting AI-generated changes
- validating correctness
- ensuring maintainability

AI should propose, draft, analyze, and assist. It should not silently own decisions.

10.3 Documentation Synchronization

When implementation changes affect documented behavior, update the relevant documentation.

Documentation updates should be included in the same commit or pull request whenever practical.

10.4 Architecture Decisions

Significant technical decisions should be recorded in ADRs even when AI helped produce the analysis.

Use ADRs for decisions involving:

- technology selection
- system structure
- database design
- API design
- deployment architecture
- security model
- major trade-offs

10.5 Single Source of Truth

Avoid creating duplicate explanations across multiple documents.

If information already exists elsewhere:

- link to it
- summarize briefly if needed
- do not copy the full content unnecessarily

10.6 PROJECT_INDEX Maintenance

`PROJECT_INDEX.md` is the primary navigation hub.

Update it when:

- major documents are created
- major documents are moved
- documentation domains are added
- authoritative documents change
- project priorities change

10.7 Future Governance Direction

A future version of the Engineering Documentation Framework may include machine-readable AI agent rules for Cursor, VS Code, Continue, and other AI tools.

Those rules should help AI assistants automatically maintain documentation consistency, cross-references, architecture decisions, and project indexes without requiring the developer to repeat the same instructions every time.

10.8 Related Documents

- AI_Philosophy.md
- Verification.md
- Security.md
- Documentation Architecture (Architecture)

10.9 Parent

- AI Engineering Handbook

11 Cost Optimization

11.1 Purpose

This document defines the cost optimization strategy for AI-assisted project work.

The objective is not to minimize API cost at all costs. The objective is to minimize total development cost, including developer time, rework, risk, and quality.

11.2 Core Principle

Use the least expensive AI that can comfortably solve the task.

A cheap tool is not actually cheap if it creates confusion, poor design, inaccurate code, or hours of manual correction.

11.3 Cost Factors

When choosing an AI tool, consider:

- model usage cost
- developer time
- task complexity
- context size
- risk of incorrect output
- security and privacy requirements
- likelihood of rework
- importance of the decision
- cost of downstream mistakes

11.4 Recommended Progression

Use this progression unless a project-specific reason justifies a different choice:

1. Local model such as Qwen 14B for routine tasks
2. Cursor + Composer 2.5 (standard) for everyday multi-file documentation and implementation
3. Low-cost cloud model for simple tasks that do not require advanced reasoning
4. Claude Sonnet for complex coding, review, debugging, and cross-file reasoning
5. GPT-5 reasoning for exceptionally difficult engineering problems
6. Codex for high-value agentic implementation tasks

11.5 When to Use Local AI

Use local AI when:

- privacy matters
- offline work is needed
- the task is routine
- the context is small
- the task involves Markdown, JSON, or simple code explanation
- cost should be near zero

11.6 When to Pay for a Stronger Model

Use a stronger cloud model when:

- the task spans multiple files
- architecture is involved
- the decision is high impact
- debugging is difficult
- cheaper models have failed

- incorrect output would create significant rework
- the model can save meaningful developer time

11.7 Avoid False Economy

Do not spend an hour trying to save a few cents.

Developer time is part of the cost equation. A more capable model is often justified when it avoids:

- architectural mistakes
- implementation churn
- repeated failed attempts
- misunderstanding of requirements
- poor documentation structure
- security mistakes

11.8 Related Documents

- AI_Philosophy.md
- AI_Decision_Matrix.md

11.9 Parent

- AI Engineering Handbook

12 Development

Internal project-building workflow, experiments, working notes, and project evolution records.

12.1 Contents

- Project Journal — evolution, experiments, and lessons learned

12.2 Related Documents

- Active Tasks *(Project README (GitHub repository root))*
- Research Backlog *(not included in PDF manual; see project repository)*
- Research (Research)

13 Project Journal

Status: Maintained **Owner:** Philippine Marching Percussion Fusion Project **Applies To:** Project evolution records and working observations **Last Reviewed:** 2026-07-13 **Review Frequency:** As needed **Authoritative:** No

13.1 Purpose

The Project Journal documents the project's evolution over time. It records working observations, experiments, and lessons learned—not authoritative specifications.

13.2 What Belongs Here

- research findings before they mature into Reference or Research documents
- experimentation notes and pilot observations
- cadence and warm-up development history
- educational observations from field testing
- lessons learned from rehearsals and performances
- identity decisions and rationale before they are formalized as ADRs

13.3 What Does Not Belong Here

- authoritative identity statements (see Identity Framework (Specifications))
- stable concept definitions (see Concept Library (Reference))
- formal research write-ups (see Research (Research))
- finished scores and audio (see `scores/` and `audio/`)

13.4 Information Lifecycle

```
Journal observation or experiment
        ↓
Research note or concept draft
        ↓
Specification, Reference, or User Guide
        ↓
Score or audio deliverable
```

13.5 Entries

Journal entries will be added below as dated sections or linked files as the project progresses.

13.5.1 2026-07-15 — Indigenous percussion research structure

Expanded Indigenous Philippine Percussion (Research) with concept profiles, performance-practice profiles, research program table, and findings log. All entries remain Outline status pending MONHS pilot validation.

13.5.2 2026-07-13 — Instructor documents for MONHS pilot

Added Instructor Introduction (User Guides) and MONHS 2026 Pilot Brief (User Guides) for music educator onboarding.

13.5.3 2026-07-13 — Musical Identity Foundation

Design-phase decisions from the project's identity foundation work were distributed into canonical EDF locations:

- Philippine Marching Percussion Identity Framework (Specifications)
- Design Principles (Specifications)
- Concept Library (Reference) (expanded)
- Research backlog *(not included in PDF manual; see project repository)*

13.6 Related Documents

- Development Overview
- Active Tasks *(Project README (GitHub repository root))*
- Research (Research)